UID:

1. **Algorithm:**

1. Sort the distances of the houses in array A[h] from smallest to largest distance from the westernmost house.
2. Initialize s = -inf, to store the last base station's distance from h.
3. Initialize a list b to later store the location of all the stations.
4. Initialize i = 0.
5. While (i < n)
    a. If ((A[i] - s) > 4):
        i. Let i be the westernmost house that is not covered by a previous base station.
        ii. Place a base station at A[i] + 4 and s = A[i] + 4.
        iii. Add s to b.
    b. i++
6. Output b.

    a. **Proof:**
        i. Correctness: Base case: n = 1; we have one house in the array. We place a base station at A[i] + 4, which would mean that house has coverage since it is within 4 miles of the station. The algorithm is effective. Inductive step: Assume that the algorithm works for some n >= 1, and it places the minimum number of base stations needed to cover all houses. Say we have n+1 houses in the array. The first n houses in the array will be covered, as explained in the base case an inductive step. If the n+1 house is within four miles of the last base station, then it is covered and the while loop will terminate since incrementing the last index will be = n. If the last house is NOT covered, the algorithm would effectively place a base station 4 miles to the right of it. By the inductive hypothesis, the algorithm correctly places the base stations to cover all the houses with as few stations as possible.
        ii. Completion: As shown by the correctness proof, we fulfill the requirement of each of the n houses having a base station within 4 miles of them. By moving past any houses covered by a past base station, we ensure that we are using as few base stations as possible.
        iii. Termination: The while loop ends if we went through all the houses in the array and are trying to access outside of it. Therefore, the algorithm will terminate.

    b. **Time Complexity Analysis:** The most efficient sorting algorithm takes O(nlogn) time. Initialization of ints and lists takes O(1) time overall. The while loop can run at most n-1 times, in case each house is more than 4 miles away from the last station and we only cover one house with each base station. At most, this is O(n).

Inside the while loop there are no significant steps contributing time and outputting b is O(1) time. This simplifies to O(nlogn) overall.

2. Algorithm:

1. Sort the degree sequences in decreasing order
2. Initialize an empty list, E, to store the graph edges
3. While (degree sequence is nonempty)
    a. If d_1 is 0:
        i. Output E (no edge added, isolated node)
    b. If d_1 is negative:
        i. Output no graph exists
    c. If d_1, the largest degree value, is greater than or equal to n:
        i. Output no graph exists (a node can't have more edges than total nodes)
    d. Let the first node d_1 be u:
        i. Remove u from the sequence
        ii. If there are less than d_i vertices left in the list
            1. Output no graph exists
        iii. For the next d_1 nodes in the list: (call them v)
            1. Add the edge (u,v) to E.
            2. Subtract 1 from v's degree in the list
    e. Re-sort the list in decreasing order, to ensure the largest degree is always at the front
4. Output E.

This algorithm will show if any graph can be constructed with the given degree and vertex sequences. The proof will show how any graph with the same degree sequence can be gradually transformed using swaps.

a. Proof:

i. Correctness:

1. Base case: Say the degree sequence is [0], where n = 1. Our algorithm would correctly output an empty list, since there are no edges. Inductive Step: Assume that the algorithm constructs a graph for any degree sequence that is n - 1 in length. When we add the nth node, the algorithm would sort the degrees in decreasing order and take the largest degree number. It will either build the necessary number of edges, or if not enough nodes exist, terminate the algorithm. If it can build the edges, it will subtract one from all of the other vertices and then repeat the process to result in either the graph existing and outputting the edge, or no graph existing. After removing a node, and decrementing the next d_1 degrees, our new degree sequence is of length n - 1. If the last d degrees were valid, all we have done is

subtract the edges and a corresponding node, so this one will also be processed correctly by the algorithm.

2. We have shown that *a* graph can be formed with the given degree sequence. Now, we must prove that the *exact* graph can be formed. Let the graph output by the algorithm be G', and the graph given in the problem be G. G' can be transformed into G through edge swaps. Say we have three nodes a, b, and c where (a,b) are connected in G but not in G', and nodes (b,c) that are not connected in G but connected in G'. Let x be a node that a is connected to in G'. By swapping the edges to form edges (a,b) and (c,x) in G' and repeating this process for any other mismatched nodes, we can transform G' into G. This is because we are not adding edges, but simply swapping them – we maintain the degree sequence and swap around the edges to ensure the degree sequence matches the vertex sequence exactly.

ii. Completeness: The conditions for the degree sequence not being valid in the first place are (1) not enough nodes left for the degrees to be fulfilled, (2) degrees become negative (3) degrees greater than or equal to n. Since we only form a graph if none of these conditions are met, our algorithm is complete in that it produces only valid graphs with the correct number of degrees and edges.

iii. Termination: The while and for loops in the algorithm are guaranteed to terminate since we only ever add at most n elements to the degree sequence. We only remove nodes from it and once the sequence is empty, the while loop terminates. Once we reach the d_i nodes in the list, the for loop ends for that while loop iteration (this cannot be more than n since if it is, our algorithm outputs that a graph does not exist).

3. Algorithm:

Since swimming is the only part that needs to be done one at a time, we cannot do anything to order them more efficiently so that swimming overall takes less time. No matter the order, the total swimming time will be the sum of all their swimming times. Therefore, we should sort by biking + running times. Those who take the longest for those segments should start first.

For my algorithm, assume the times are given in 3 arrays, S, B and R.

1. Sum all the running and biking times of each participant and assign it to an array "total". (assuming numpy arrays).
2. Sort the participants by the sum of running and biking time in descending order.
3. Output total. This is the start order that produces the most optimal triathlon time.

a. Proof:

i. Correctness: Say we have Participant A and Participant B, with times s_A, s_B, b_A, b_B, r_A, and r_B. Say A is assigned to go before B by our descending order algorithm, meaning that $b_A + r_A > b_B + r_B$. We are trying to minimize the sum of $s_A + s_B$ + the last athlete's running and biking time. Our algorithm would mean we add b_B and r_B since they are last.
Suppose, for the sake of contradiction, that there is a more optimal ordering, meaning there is a better way to sort the athletes in step 1. Instead of descending order, we do ascending order. This means our total time is $s_A + s_B + b_A + r_A$ since A would go last. By our prior inequality, $b_A + r_A > b_B + r_B$ so this total time is larger than the time our algorithm's sorting would take. Our initial descending order sorting is the most optimal. The assumption that a better ordering exists is wrong.

Having an athlete that takes longer to run and bike to go later in the race and switching them with a faster one would extend the overall time of the race. The slower athlete is taking longer and has less of an overlap with other runners, meaning they will extend the time past the overall time of the last biker/runner faster than them. We do not take swimming into account since swimming time cannot be minimized since they swim one at a time.

ii. Completeness: As long as we have every participant's biking and running times, each of them will be assigned a spot in the final array. The way that the algorithm works also ensures that they have time to complete their full race, since the slowest runners and bikers start first. We also maximize overlap in biking and running this way, since participants with shorter times can later overlap with the longer time participants.

iii. Termination: There are no loops in our algorithm. The algorithm will terminate after the times have been summed and then sorted.

b. **Time Complexity Analysis:** Step 1 takes O(n) time. The most efficient sorting algorithm would take O(nlogn) time (quick sort). Therefore, the total time complexity is O(nlogn).

4.

a. This claim is false in stating that we can ONLY get a valid schedule if $b_i <= r*t_i$. Say that we have three streams and for one stream we have to flip the inequality, so that $b_i > r*t_i$, we could still make a schedule.

$b_1 = r - 1$	$t_1 = 1$	$r - 1 <= r$
$b_2 = 2r$	$t_2 = 2$	$2r <= 2r$

$b_3 = r + 1$	$t_3 = 1$	r + 1 > r (not satisfying condition)

At t = 1, send the first stream,	r - 1 <= r
At t = 2, send half the second stream,	r - 1 + r <= 2r
At t = 3, send the rest of the second stream,	r - 1 + 2r <= 3r
At t = 4, send the last stream,	r - 1 + 2r + r + 1 <= 4r.

Even though the fourth stream did not fulfill the inequality on its own, we could still make a schedule to fulfill the requirement overall.

b. Algorithm:

- **i.** Sort the set of streams by b_i/t_i, in ascending order.
- **ii.** Initialize a variable elapsed_time = 0, to store the current time that has passed.
- **iii.** Initialize a variable total_bits = 0, to store the current bits that have been processed.
- **iv.** For each stream i in the set
    1. elapsed_time = elapsed_time + t_i
    2. total_bits = total_bits + b_i
    3. if total_bits > r * elapsed_time
        a. Output that no valid schedule exists
- **v.** Output that a valid schedule exists.

c. Proof:

- **i.** Correctness: Sorting by rate will lead to a valid schedule, if any exists. This is because the main point is to not exceed r. You can reframe the inequality to b/t <= r. We are sending the smallest amount of bits per second first as we can. So if adding a higher rate stream violates the schedule, there is no way we can reorder it to get a valid schedule. If it violates the current condition at time t, it will also violate the condition at time t-1, since lowering the denominator means an even higher number, so it will not become less than or equal to r.
- **ii.** Base case: Assume that the condition is met for a single stream and we can find a schedule. Inductive Step: Assume that the algorithm is valid for any n - 1 streams. Say we add an nth stream, and we have already added the first n -1 streams. If adding b_n and t_n to our total current bits and time violates the condition, then a valid schedule does not exist. If not, a valid schedule does exist.
- **iii.** Completeness: We are ordering each stream and including it in the schedule. The only requirement is that the overall condition is not violated, so we check that for every stream we add, we do not violate that condition.
- **iv.** Termination: The for loop will terminate since we only process each stream at most once, so n streams total. Overall, there are no other loops

and once we either violate the condition or reach the end of the loop, we output that a valid schedule does or does not exist.

d. **Time Complexity Analysis:** Finding the rates is time O(n). Sorting the streams is most efficient at O(nlogn). The for loop will take at most O(n), since we process each stream in the set once. The total time complexity is O(nlogn).

5. Algorithm:

1. Negate all bandwidths or b_e. Let these bandwidths serve as edge weights for the graph.
2. Pick any vertex s in G, let that be the starting node.
3. Initialize A as an n length array where A[s] = 0 and A[v] = infinity for all v that are not s.
4. Initialize P as a priority queue.
5. Add (A[v], v) to P for all nodes v.
6. Initialize tree T as a graph containing no nodes and no edges.
7. Initialize Parent as an n-size array.
8. While (P is nonempty)
    a. Pop the minimum element (A([v],v) from P.
    b. Add node v to T
    c. If v is not s, add (parent[v],v) as an edge to T.
    d. For each neighbor x of v still in P:
        i. if (w(v,x) < A[x])
            1. Set A[x] = w(v,x)
            2. Update the key of x in P to be the new value of A[x]
            3. Set Parent[x] = v.
9. Output T.

a. Proof:

i. Correctness: As was shown in class, Prim's algorithm will produce a minimum spanning tree on any connected graph with weights when they are distinct. This can be extended by what was proved in the textbook – the algorithms also do the same with non-distinct edge weights. Therefore, we can conclude that for any connected graph, a spanning tree will exist. Since the problem states the graph is connected and bandwidths are distinct, our algorithm fulfills the requirements to find a spanning tree. This algorithm finds a minimum spanning tree. But since we negated the bandwidths, the minimum spanning tree will actually have the maximum spanning tree of the original graph (since the absolute value of the smallest negative numbers is largest). By negating the bandwidths and then finding a minimum spanning tree we are finding the maximum spanning tree, which ensures that the bottleneck rate of the path between two nodes is the best achievable rate possible.

ii. Further, the proof that Prim's algorithm is correct to produce a minimum spanning tree was shown in class. Just like with an MST, we add the smallest possible weight we can one at a time, with the negation we are

adding the highest possible weight – meeting the maximization of the bottleneck rate requirement.

iii. Completeness: If a valid minimum spanning tree exists, the algorithm outputs it and the absolute value of all the edges is the maximum spanning tree. If no tree exists, T will be empty. However, we proved in the correctness proof that Prim's will result in an MST in a connected, weighted graph.

iv. Termination: The only loop is in the while loop, and it will end when P is empty, or every node is processed and added to the tree. Therefore, we know the algorithm terminates.

b. Time Complexity Analysis:

- Negating the bandwidths takes O(m), if there are m edges.
- The rest just follows Prim's algorithm which as explained in the textbook, was time O(mlogn), where m is the number of edges and n is the number of nodes.
- Overall time: O(mlogn)

6.

a. True. When we use Dijkstra's algorithm to find a shortest path, we repeatedly pick the vertex with the lowest distance that is unvisited. Say you have two nodes, a and b, that you use to get from s to t, (s → a → b → t). Say s is connected to both a and b, but a is picked first since it is less weight to get to. Even if you square all of the edge weights in the graph, if $s \rightarrow a < s \rightarrow b$, then $(s \rightarrow a)^2 < (s \rightarrow b)^2$. The relationship between edge weights remains the same, and thus when picking smallest edge weights, we would still go in the same order, resulting in the same shortest path. There are exceptions to this if we had 0's or multiple of the same edge weights, but since we have distinct edge weights, we are sure that the square of the weights will maintain the current relationship between weights. Thus, we'll pick nodes in the same order.

b. True. When we use Prim's algorithm to find an MST, we repeatedly pick the cheapest edge that has not yet been added to the spanning tree. If all the edges are distinct, we are guaranteed to have some ordering of the weights, where $e_i < e_j < ... e_n$. When you square the weights, this same ordering will be maintained since the square of distinct weights will have the same relationship. Therefore, the MST of G will be maintained in G', where we square the weights. The weight of any minimum spanning tree in G will be the same in G', just squared, so the smallest is still the smallest.